ONLINE EXAM PORTAL

Software Requirements Specification, Software Architecture and Design Specification, and Software Test Plan

Field	Details
Project ID	OEP-2026-13
Team	Team 13 (G-13)
Course	UE24CS341A Software Engineering, V Semester
Technology baseline	MongoDB, Express.js, React.js, Node.js; JWT authentication
Document version	1.1, 03 October 2026
Basis	Team 13 Deliverable 1 and finalized Deliverable 2; shared SRS, SAD and STP templates

Team members

Student	USN
Saikavin S	PES1UG24CS402
Samarth Kotian	PES1UG24CS406
Samrudh C Patil	PES1UG24CS417
Satyam Goyal	PES1UG24CS424

Document status: requirements and design baseline prepared for team verification. Test cases describe planned validation; no test execution result or approval is asserted.

Revision and approvals

Version	Date	Change	Approval
1.0	September 2026	Deliverable 2 requirements baseline	As recorded in D2
1.1	03 Oct 2026	Combined SRS, SAD and STP draft	Team review pending

Approval roles: team lead, faculty evaluator, and test lead. Signatures and dates to be completed by the respective approvers.

Part I Software Requirements Specification

1. Introduction

1.1 Purpose

This SRS defines the Online Exam Portal requirements that developers must implement and testers must verify. It preserves Team 13 Deliverable 2 requirement identifiers so design and test evidence can be traced without changing the agreed project scope.

1.2 Scope

The portal supports student and administrator login, a categorized multiple-choice question bank, scheduled exams, randomized questions, timed attempts with auto-save, objective scoring, student results, and administrator reports. AI or webcam proctoring, subjective grading, ERP integration, native mobile apps, and live surveillance are outside scope.

1.3 Audience and references

Audience: Team 13 developers and testers, course faculty, and examination-cell stakeholders. References: Team 13 Deliverable 1 synopsis; OEP-2026-13 Deliverable 2 Final; the course SRS, SAD and Test Plan templates; Software Engineering mini-project guidelines.

1.4 Definitions

API: application programming interface. JWT: JSON Web Token. RBAC: role-based access control. Attempt: one student participation in an exam. Auto-save: persistence of answers during an active attempt. RTM: requirements traceability matrix.

2. Overall description

2.1 Product perspective

A browser client communicates with a REST API. The API authenticates users, enforces permissions, runs exam and evaluation logic, and stores users, questions, exams, attempts and results in MongoDB. The system is a standalone application.

2.2 Product functions and roles

Students register or log in, discover available exams, complete timed attempts, and see their own results. Administrators (including faculty in this two-role baseline) maintain questions and exams and view reports. Both roles use a responsive browser interface.

2.3 Environment, constraints and assumptions

The planned implementation uses React, Node.js/Express, MongoDB, JavaScript and JWT. A functioning browser, network and database are assumed. Only objective multiple-choice questions are automatically scored. The exam window and duration are configured by an administrator. Production traffic uses HTTPS. Docker, GitHub and Jenkins are planned for deployment and delivery evidence.

3. External interface requirements

Interface	Required behavior
Login/registration	Validate fields; show actionable failures; establish a role-appropriate session.
Student dashboard	Show exams available in the current window and links to results.
Exam screen	Show question, options, remaining time, navigation, answer and review state.
Admin dashboard	Provide question-bank, exam scheduling and reporting functions.
REST API	Exchange JSON with the React client; authenticate secured requests with JWT.
Database	Persist user, question, exam, attempt and result records with appropriate relationships.
Communication	Use HTTPS in deployment; handle temporary save failures with retry and recovery.

4. System features and functional requirements

The following stable IDs come from the finalized Team 13 Deliverable 2. Unless stated otherwise, acceptance is verified through a positive case and a relevant invalid or unauthorized case.

ID	Priority	Requirement	Acceptance check
FR-001	High	Register and log in with a unique email.	Valid credentials open a session; duplicate email and wrong password fail.
FR-002	High	Enforce Student versus Administrator access.	Student requests to admin routes are denied by the API.
FR-003	High	Let administrators create, edit, disable and categorize MCQs.	Valid changes persist; disabled questions are excluded from new selection.
FR-004	High	Let administrators create exams with duration, window and selection criteria.	Valid configuration saves; invalid dates or insufficient questions fail.
FR-005	High	Randomly select and shuffle questions per attempt.	Selection obeys criteria and persists an individual question order.
FR-006	High	Show currently available exams to authenticated students.	Only exams in the active window appear as startable.
FR-007	High	Provide timed attempts with remaining time shown.	Timer starts from server-recorded attempt start and displays remaining time.
FR-008	Medium	Allow navigation and mark for review.	Student can revisit questions; review state survives navigation.
FR-009	High	Automatically save selected answers during an active exam.	Saved answer can be restored after reconnecting.
FR-010	High	Automatically submit at the allocated time limit.	Expired attempt becomes submitted and rejects later changes.
FR-011	High	Prevent two active attempts by one student for the same exam.	Concurrent start request returns the existing attempt or a conflict.
FR-012	High	Evaluate objective answers and calculate total score on submission.	Correct, wrong, unanswered and configured penalty rules yield expected score.
FR-013	High	Generate and store one final result for each completed attempt.	Result is persisted and repeated submission does not duplicate it.
FR-014	Medium	Show a student only their own history, scores and performance.	Another student’s result is inaccessible.

ID	Priority	Requirement	Acceptance check
FR-015	Medium	Provide exam-wise and student-wise administrator reports.	Authorized reports agree with stored submitted results.

5. Nonfunctional requirements

ID	Category	Requirement	Evidence
NFR-001	Performance	95% of exam-attempt API requests finish within 500 ms under the agreed test load.	Load-test percentile.
NFR-002	Availability	Target 99.9% availability during active exam windows in the intended deployment.	Deployment monitoring review.
NFR-003	Usability	A student starts an available exam in at most 3 clicks from the dashboard.	Task walkthrough.
NFR-004	Reliability	Auto-saved state restores within 1 minute after reconnection.	Recovery scenario.
NFR-005	Maintainability	Evaluation logic is isolated and has at least 80% automated unit coverage.	Coverage and design review.
NFR-006	Testability	Question selection and scoring can run without the web framework.	Independent unit tests.

5.1 Security objectives and requirements

Objectives: (1) protect credentials and exam data from disclosure; (2) preserve integrity of answers, submissions and scores; (3) ensure each role sees only authorized data and functions.

ID	Security requirement	Verification
SEC-001	Hash passwords with bcrypt before storage.	Stored credential review and login test.
SEC-002	Reject unauthorized access to role-protected APIs.	Student/admin cross-role tests.
SEC-003	Validate and sanitize input against NoSQL injection and XSS.	Malicious payload tests.
SEC-004	Expire JWTs and validate each secured request.	Invalid and expired token tests.
SEC-005	Do not expose questions to a student before their active attempt.	API response inspection.
SEC-006	Log failed logins and unauthorized API attempts.	Audit-log inspection.

6. Quality attributes and acceptance

Acceptance requires all high-priority functional requirements and security checks to pass, no critical defect to remain open, and the RTM to show evidence for each baseline requirement. Submitted attempts must be immutable through normal operations. Unanswered questions score zero; negative marking follows the exam setting. No physical safety function applies to this software-only system.

7. UML use-case diagrams

Figure 1. Student use cases. The attempt and result functions require authentication.

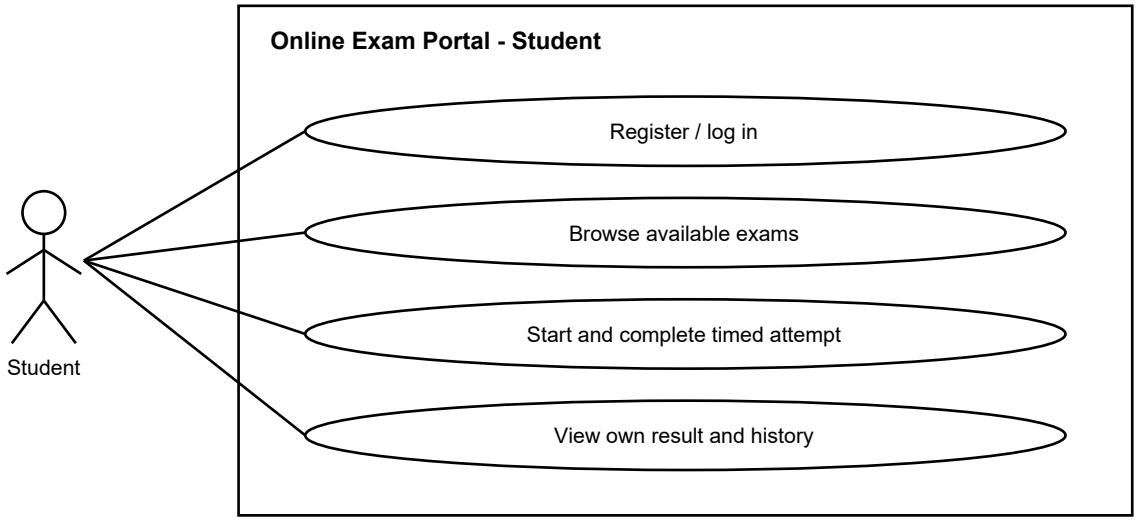
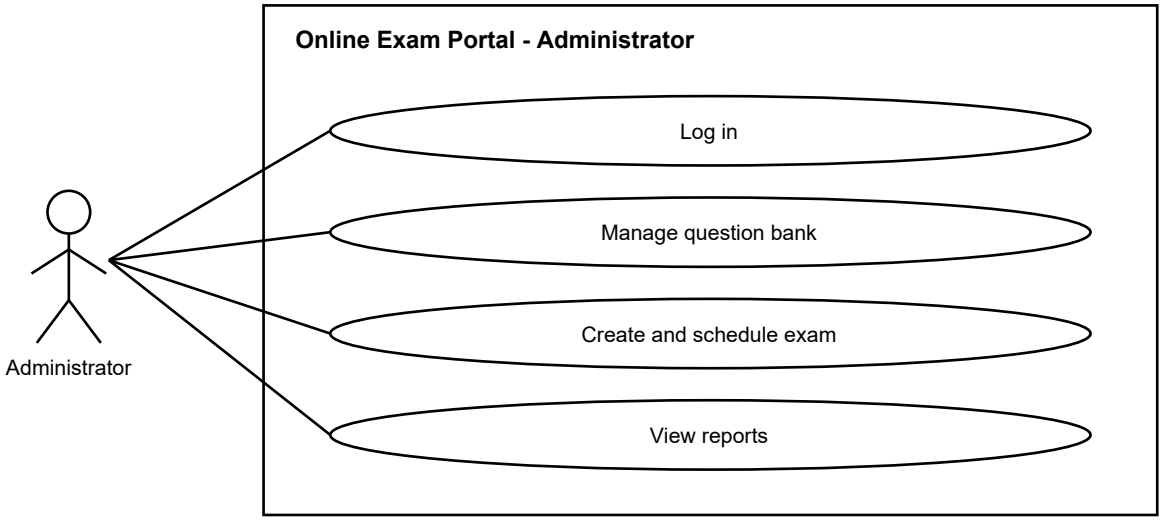


Figure 2. Administrator use cases. Faculty is included in the Administrator role.



8. Requirements traceability matrix

Requirement IDs	Architecture / design	Code reference	Test IDs
FR-001, FR-002, SEC-001, SEC-002, SEC-004, SEC-006	Authentication/RBAC	server/src/auth.js; app.js	TC-01, 02, 19, 20, 29
FR-003, SEC-003	Question service	server/src/app.js; models.js	TC-03, 04, 21
FR-004, FR-005, FR-006, SEC-005	Exam service/selector	server/src/app.js; domain.js	TC-05 to 08, 22
FR-007 to FR-011	Attempt service/UI	server/src/app.js; client/src/main.jsx	TC-09 to 14
FR-012, FR-013	Scoring/result service	server/src/domain.js; app.js	TC-15 to 17
FR-014, FR-015	Result/report service	server/src/app.js; client/src/main.jsx	TC-18, 23
NFR-001 to NFR-006	API, UI, domain	server/src; client/src	TC-24 to 28

Other data requirements: unique email; a stored question order and answers for each attempt; one result per completed attempt; records for submitted attempts remain unchanged. A final retention period has not been specified by the team and must be set before deployment.

Part II Software Architecture and Design Specification

Revision history and approvals

Version	Date	Change	Approval
1.0	03 Oct 2026	Architecture and design based on finalized D2	Team review pending

1. Introduction

1.1 Purpose and scope

This specification defines the proposed component boundaries, interactions, data structures, APIs and security design for the Online Exam Portal. It covers the baseline functions in SRS Part I and provides interfaces for implementation and testing.

1.2 Audience and definitions

Audience: developers, testers, faculty evaluators and maintainers. API, JWT, RBAC, MCQ and RTM have the meanings defined in the SRS.

2. Document overview

Section 3 describes architecture and its relationship to requirements. Section 4 specifies interactions and component interfaces. The SRS is the controlling requirements baseline and the STP supplies planned verification.

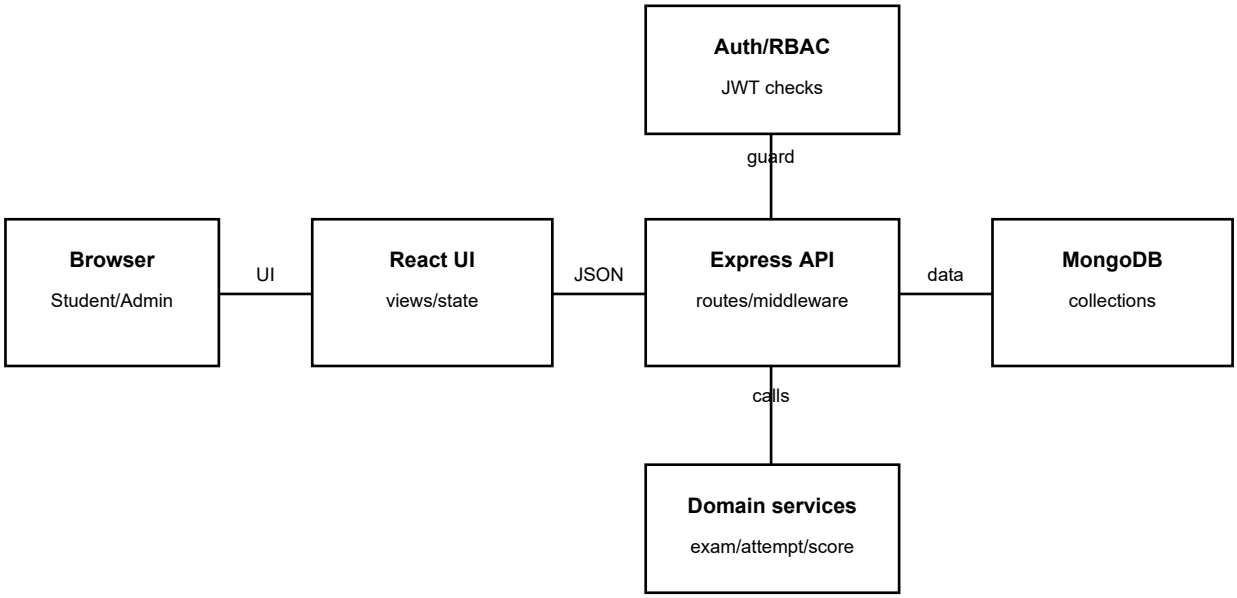
3. Architecture

3.1 Goals, constraints and stakeholders

Goals are secure separation of student and administrator functions, reliable timed attempts, reproducible scoring, and modular testable code. Constraints are the MERN stack, objective questions and a browser-based client. Students need a clear uninterrupted exam flow; faculty need controlled exam configuration; testers need observable, isolated behavior.

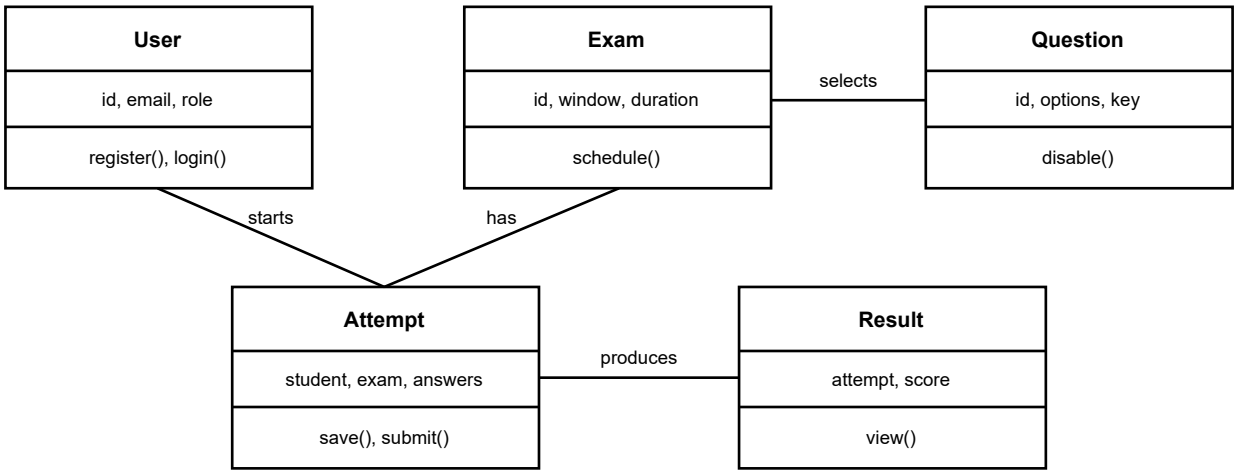
3.2 UML component diagram

Figure 3. Proposed layered components and their principal dependencies. API guards run before domain services.



3.2.1 UML class diagram

Figure 4. Main domain classes and relationships; fields are abbreviated for readability.



3.3 Component descriptions

Component	Responsibility	Key data / interface
React UI	Routes for login, dashboards, exam and reports; local answer and timer display.	REST JSON; no answer key before submission.
Express API	Routes, validation, error translation and request logging.	HTTP endpoints under /api.
Auth/RBAC	Password verification, JWT issue/validation and permission checks.	User identity, role, token claims.
Question/Exam services	Question lifecycle, exam validation and selection/shuffling.	Questions, exams, attempt question snapshot.
Attempt service	Start, auto-save, deadline and finalization state machine.	Attempt record and server time.
Scoring/Results	Pure scoring rules, result persistence and reports.	Submitted attempt, answer key, score.
MongoDB	Persist users, questions, exams, attempts and results.	Indexes and unique constraints.

3.4 Pattern, stack and rationale

A layered modular monolith separates React presentation, Express HTTP concerns, domain services and MongoDB persistence. This keeps the four team work areas distinct while allowing scoring and selection to be unit tested without HTTP. The technology baseline is JavaScript, React, Node.js, Express and MongoDB. JWT is the planned session mechanism.

3.4.1 Module ownership

Module	Main responsibility	Planned owner
Authentication / administration	Registration, login, JWT and RBAC	Saikavin S
Question bank / scoring	MCQ lifecycle, selection, evaluation	Samarth Kotian
Exam / attempt	Scheduling, attempt state and auto-save	Samrudh C Patil
Student UI / performance	Exam interface, history and usability	Satyam Goyal

3.5 Data design and invariants

Collection	Minimum fields	Invariant
users	id, name, email, passwordHash, role, status	email unique; role is Student or Administrator
questions	id, text, options, correctOption, category, status	correctOption references a valid option
exams	id, title, durationMinutes, startTime, endTime, category, questionCount, negativeMarking	startTime before endTime; duration positive
attempts	id, studentId, examId, question snapshot, answers, status, deadline	one active attempt per student/exam; submitted is immutable
results	id, attemptId, score, submittedAt	one result per submitted attempt

Use server time to decide exam eligibility and deadline. Persist the selected question order when an attempt begins. Final submission and result creation should be atomic or idempotent so retries cannot create duplicate results. Do not return correctOption in student attempt payloads.

3.6 Risks and mitigations

Risk	Mitigation
Lost answers during network failure	Auto-save acknowledgment, retry queue and restored server state.
Client timer tampering	Server-recorded deadline controls submission.
Duplicate concurrent attempts/results	Unique constraints and idempotent start/finalize operations.
Question leakage	Separate student response DTO from stored question/answer key.
Unspecified retention period	Set and approve period before live deployment.

3.7 Requirement traceability

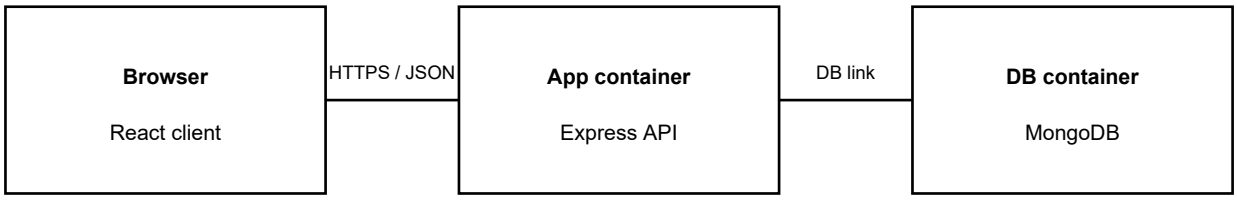
Architecture element	Requirement IDs
Auth/RBAC	FR-001, FR-002, SEC-001, SEC-002, SEC-004, SEC-006
Question and exam services	FR-003 to FR-006, SEC-003, SEC-005
Attempt service and exam UI	FR-007 to FR-011, NFR-001, NFR-003, NFR-004
Scoring/results/reports	FR-012 to FR-015, NFR-005, NFR-006

3.8 Security architecture

Threat	Control	Verification
Spoofing	bcrypt password hashes; JWT validation and expiry.	SEC-001, SEC-004
Privilege escalation	Server-side RBAC on every protected endpoint.	SEC-002
Injection and XSS	Input schema validation, safe database queries, output escaping.	SEC-003
Question disclosure	Student API omits answer keys and pre-start content.	SEC-005
Repudiation	Timestamped failed-login and denied-access audit records.	SEC-006
Tampering with deadline/score	Server time, immutable submission, server-side scoring.	FR-010, FR-012

3.9 UML deployment diagram

Figure 5. Local container deployment. Production requires HTTPS termination and monitoring.



Docker Compose is supplied for a local reproducible deployment.

4. Design

4.1 Design overview

The API validates each request, resolves the authenticated user, calls a domain service and returns a safe response. Domain services enforce schedule and state rules. Database adapters isolate persistence from scoring and selection logic. The sequence diagrams show the two main workflows.

4.2 UML sequence diagrams

Figure 6. Start and complete student attempt

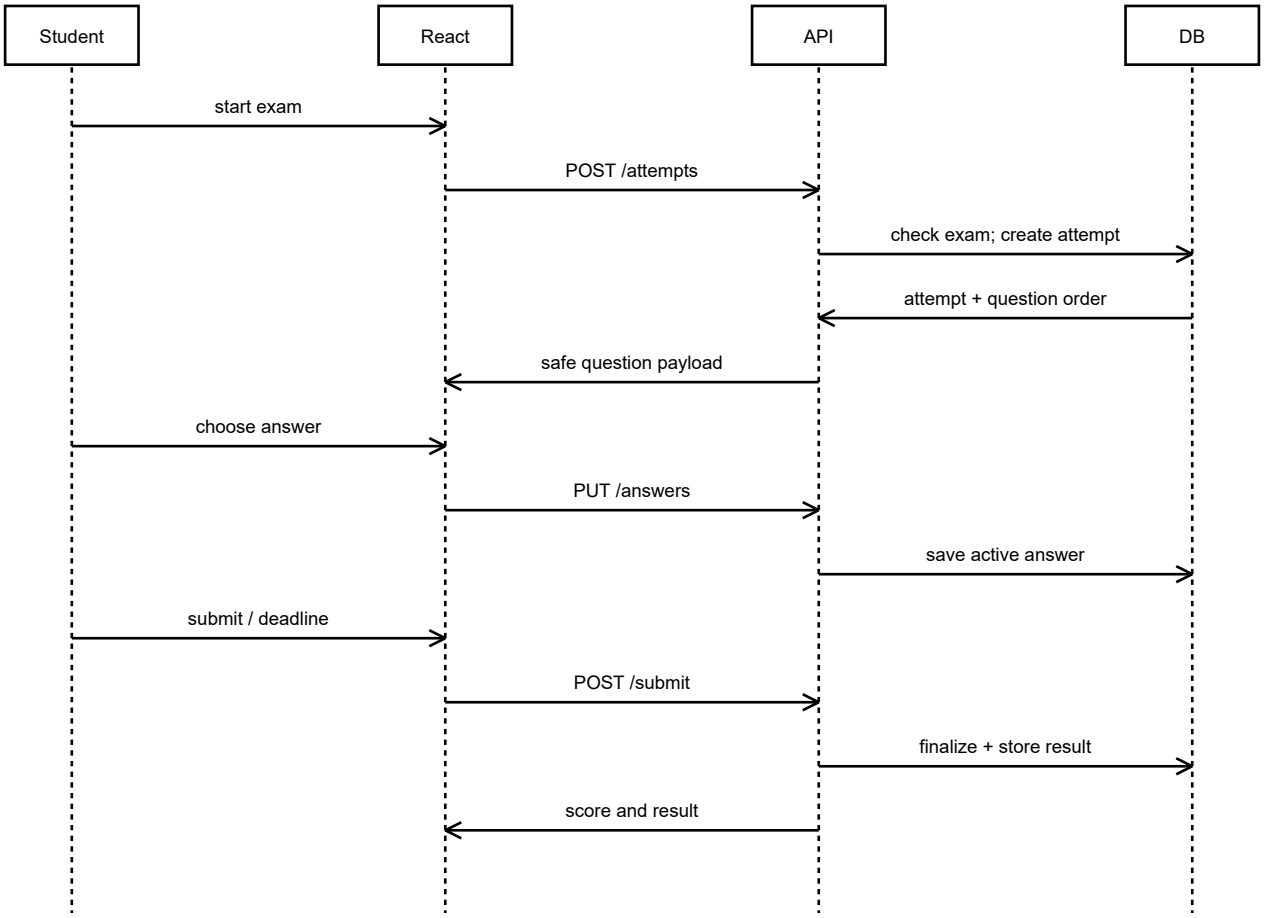
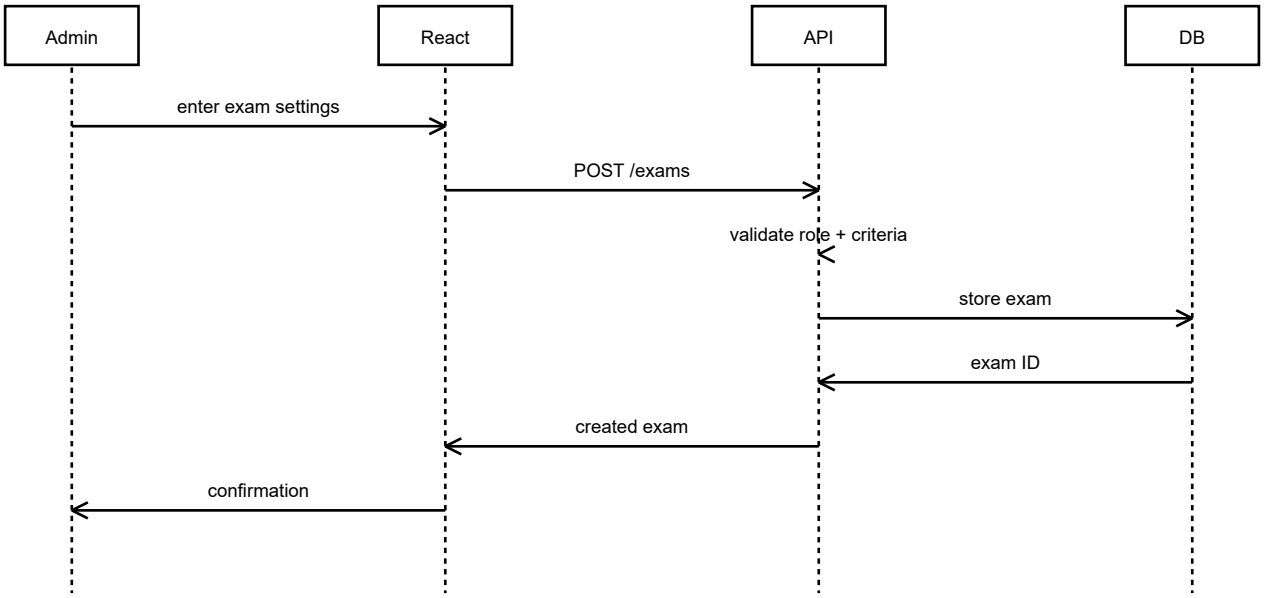
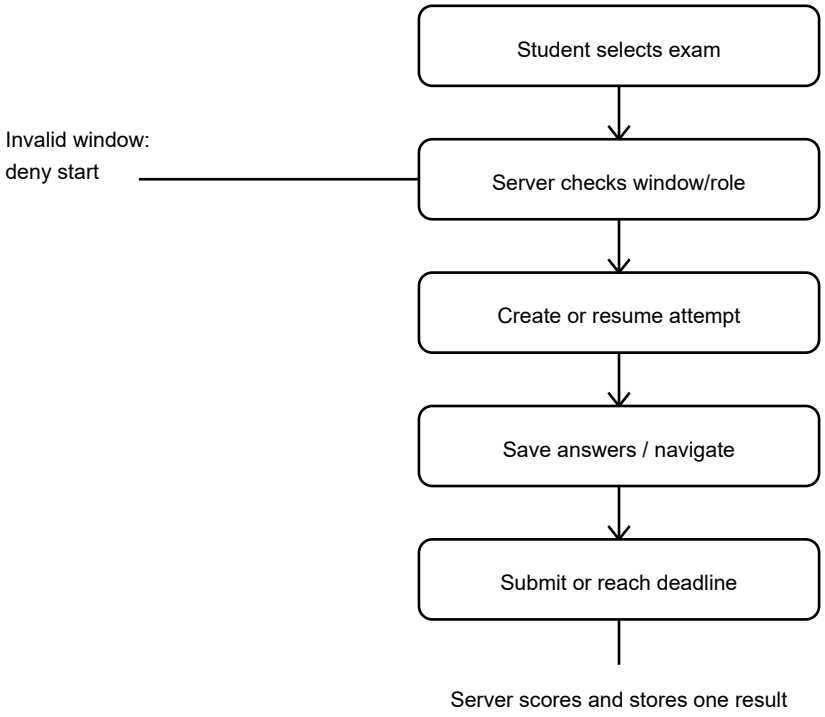


Figure 7. Administrator creates and schedules exam



4.2.1 UML activity diagram

Figure 8. Student attempt workflow and invalid-start branch.



4.3 API design

Proposed JSON interfaces for the React client and Express API. Exact route names may change with the repository; behavior, roles and error cases are contractual.

Method / route	Role	Request and successful response	Errors
POST /api/auth/register	Public	{name,email,password} -> 201 {userId}	400 invalid; 409 duplicate

Method / route	Role	Request and successful response	Errors
POST /api/auth/login	Public	{email,password} -> 200 {token,role}	401 invalid credentials
POST /api/questions	Admin	{text,options,correctOption,category} -> 201 {id}	400 invalid; 403 forbidden
POST /api/exams	Admin	{title,durationMinutes,startTime,endTime,category,questionCount} -> 201 {id}	400 invalid; 403 forbidden
GET /api/exams/available	Student	No body -> 200 [{examId,title,window}]	401 unauthenticated
POST /api/exams/{id}/attempts	Student	No body -> 201 {attemptId,deadline,questions}	403 outside window; 409 active attempt
PUT /api/attempts/{id}/answers	Student	{questionId,optionId,review} -> 200 {savedAt}; optionId optional for review only	404 owner mismatch; 409 submitted
POST /api/attempts/{id}/submit	Student	No body -> 200 {resultId,score}	403 owner mismatch; 409 invalid state
GET /api/results/me	Student	No body -> 200 [own results]	401 unauthenticated
GET /api/reports/exams/{id}	Admin	No body -> 200 {summary,students}	403 forbidden

4.4 Error handling, logging and monitoring

Use a consistent error body {code,message,requestId}. Return 400 for validation, 401 for missing/invalid authentication, 403 for authorization, 404 for unknown resources and 409 for state conflicts. Never log passwords, JWTs, answer keys or full student answers. Log request IDs, failed logins, forbidden access, save failures, finalization failures and API latency; monitor failed-save rate and exam-window availability.

4.5 UX design

Use clear labels, readable text and keyboard-accessible navigation. Show save status and remaining time on the exam screen. Confirm manual submission; explain that automatic submission occurs at the deadline. Use concise errors that identify a recoverable action. Avoid exposing answer keys in the student UI before policy permits release.

4.6 Open issues and next steps

Confirm the actual repository routes and model names, the exam load used for NFR-001, the deployment used for availability evidence, the data-retention period, and the result-release policy. Update this design to match implementation before claiming final approval.

5. Appendices

Glossary: SRS Part I, section 1.4. Related documents: Team 13 Deliverables 1 and 2, SRS Part I, STP Part III. Diagram notation: boxes denote components or participants; arrows denote request and data flow.

Part III Software Test Plan

1. Introduction

Purpose: define the planned verification of the Online Exam Portal against the SRS and SAD. Scope includes browser flows, REST APIs, persistence, security and measurable quality targets. This plan does not report executed results. References: SRS Part I, SAD Part II and Team 13 Deliverable 2.

2. Test items

Authentication/RBAC; question bank; exam configuration and discovery; attempt engine and timer; auto-save; scoring and results; student history; administrator reports; REST API and MongoDB integration.

3. Features to be tested

Every FR-001 to FR-015, NFR-001 to NFR-006 and SEC-001 to SEC-006 is in scope. The matrix in section 13 links each requirement to at least one planned case.

4. Features not to be tested

AI proctoring, webcam surveillance, subjective grading, ERP integration and native mobile applications are excluded by the requirements baseline. Vendor internals of browsers and MongoDB are outside the team test scope.

5. Test approach and strategy

Unit tests exercise pure question selection, scoring and validation rules. Integration tests exercise API, database, authorization and attempt state transitions. System tests run the main student and administrator journeys. Regression tests rerun after changes. Performance, recovery, usability and security checks address the nonfunctional baseline. Test data uses synthetic accounts and exams only.

Entry: stable build, seeded test database, configured time window and known test accounts. Exit: all planned high-priority cases executed, every critical/high defect fixed and retested, no critical security finding open, and RTM evidence recorded. Unexecuted cases remain explicitly marked Pending.

5.1 Security validation

Test password storage, role checks, token expiry, malicious input handling, question secrecy and audit logging. Inspect student API responses for answer keys. Verify controls on the server, not merely by hiding UI buttons. Record request/response evidence with secrets redacted.

6. Test environment

Item	Planned setup
Client	Modern desktop browser; responsive viewport for usability walkthrough.
Application	React client and Node.js/Express API from the team repository.
Data	Isolated MongoDB test database with Student and Administrator accounts.
Tools	Jest/Testing Library or equivalent, API client, load test tool, coverage reporter and defect tracker.
Fixtures	Active, future and expired exams; enough MCQs for randomization; valid, invalid and expired JWTs.

7. Test schedule

Stage	Trigger / output
Design	After SRS/SAD review: freeze IDs, fixtures and expected outcomes.
Unit and API checks	On every implementation change and CI build.
System and security checks	After integration build on a seeded test environment.
Load, recovery and usability	After core attempt workflow is stable.
Regression and sign-off	Before final release; publish results, defects and RTM status.

8. Test deliverables

This plan, test data and scripts, CI test/coverage reports, API and system execution logs, defect records, security findings, load metrics, final test summary and updated RTM.

9. Roles and responsibilities

Owner	Planned responsibility
All four team members	Review requirements and execute cross-module system tests.
Saikavin S	Authentication, RBAC and security cases.
Samarth Kotian	Question bank, selection, scoring and result cases.
Samrudh C Patil	Exam configuration, scheduling, attempt and API reliability cases.
Satyam Goyal	Student interface, navigation, history and usability cases.

10. Risks and mitigation

Risk	Mitigation
Unstable build	Run smoke checks before full suite; triage blockers.
Time-sensitive exam tests are flaky	Use a controllable clock and fixed fixtures.
Insufficient question data	Seed deterministic banks of known categories and answer keys.
No deployed uptime history	Report target as unverified until monitoring evidence exists.
No agreed load profile	Set concurrency and data volume with faculty before NFR-001 measurement.

11. Assumptions and dependencies

The team provides a runnable build, test accounts, database access and finalized behavior for score policy, result release and retention. Jenkins/Docker evidence depends on actual pipeline configuration. Test cases below are specifications, not pass claims.

12. Suspension and resumption

Suspend affected tests when the build cannot start, the database is unavailable, or a blocker prevents a required journey. Resume after the cause is fixed, the build passes a smoke test and fixtures are restored. Keep unaffected unit and design-review work moving.

13. Test cases and traceability

Status of every case: Planned. Unless specified, precondition is a seeded test environment and a signed-in user of the named role. Capture actual outcome, defect ID and date during execution.

Case	SRS IDs	Steps / scenario	Expected outcome
TC-01	FR-001	Register unique student; try duplicate email and wrong login.	Registration/login succeeds once; invalid attempts fail.
TC-02	FR-002, SEC-002	Call admin API with student token.	403; no data change.
TC-03	FR-003	Admin creates, edits and disables categorized MCQ.	Changes persist; disabled MCQ excluded from new attempts.
TC-04	FR-003	Student calls question mutation API.	403; question unchanged.
TC-05	FR-004	Create valid exam; try invalid window/duration.	Valid exam saved; invalid data rejected.
TC-06	FR-005	Start separate attempts from a bank larger than exam size.	Each order saved; all selected IDs satisfy criteria; no duplicate within attempt.
TC-07	FR-006	Open dashboard before, during and after exam window.	Only current exams are startable.
TC-08	FR-004, FR-006	Try starting outside valid window.	Start blocked; no active attempt created.
TC-09	FR-007	Start timed exam and inspect server deadline/UI timer.	Displayed time tracks server deadline.
TC-10	FR-008	Navigate away/back and mark item for review.	Answer and review marker remain.
TC-11	FR-009	Select answer; inspect save acknowledgment and stored attempt.	Latest answer persisted.
TC-12	FR-009, NFR-004	Disconnect after save; reconnect.	Saved state restored within 1 minute.
TC-13	FR-010	Advance to deadline without manual submission.	Attempt auto-submits; later answer write rejected.
TC-14	FR-011	Send two near-simultaneous start requests.	At most one active attempt exists.
TC-15	FR-012	Submit known correct, wrong and unanswered answers.	Score matches answer key and zero for unanswered.
TC-16	FR-012	Submit exam with configured negative marking.	Wrong-answer penalty matches configured rule.
TC-17	FR-013	Submit and retry the same request.	One durable result; score unchanged.
TC-18	FR-014	Student requests own and another student result.	Own history shown; other result denied.
TC-19	SEC-001	Inspect stored user and authenticate.	bcrypt hash stored; plaintext absent.
TC-20	SEC-004, SEC-006	Use expired/invalid JWT; fail login; inspect logs.	Requests rejected; redacted events recorded.
TC-21	SEC-003	Send NoSQL operator and XSS payloads in inputs.	Invalid input rejected or safely encoded; no query bypass.
TC-22	SEC-005	Inspect student exam APIs before active attempt.	Question text/key not exposed early; answer key never in attempt payload.
TC-23	FR-015	Admin requests exam/student reports for known results.	Counts and scores match database; student route denied.

Case	SRS IDs	Steps / scenario	Expected outcome
TC-24	NFR-001	Measure attempt API latency at agreed concurrent load.	95th percentile at most 500 ms.
TC-25	NFR-002	Review monitoring during active exam window.	Observed availability reaches 99.9% target; otherwise fail/pending.
TC-26	NFR-003	Observe new student starting available exam.	At most 3 clicks from dashboard.
TC-27	NFR-005	Run unit coverage for evaluation logic.	At least 80% coverage with isolated logic.
TC-28	NFR-006	Run selector/scorer unit tests without Express/MongoDB.	Tests pass independently.
TC-29	SEC-006	Call forbidden API repeatedly and inspect audit trail.	Each denied attempt recorded without token/secret.

14. Test metrics and reporting

Track executed/planned cases, pass/fail/block counts, requirement coverage, open defects by severity, defect age, API 95th-percentile latency, recovery time and evaluation coverage. Publish a final summary with environment, build/commit ID, actual results, evidence links and unresolved risks.

15. Approvals

Role	Name	Signature / date
Test lead	To be assigned	Pending
Development lead	To be assigned	Pending
Faculty / stakeholder	To be assigned	Pending